- The problem, intended user or client, intended use, and target outcome

- **Problem:** A rider notices something wrong with the motorcycle but cannot tell which fault it is, whether the bike is still safe to ride

  This project builds a diagnostic agent. The input is a photo, model year, symptoms, and mileage for a 2013–2018 Honda CB500F.

- **The intended user:** CB500F owner.
- **The intended use:** it is a pre-ride reference, not a certified inspection.
- **The target outcome:** lists only causes supported by the photo, the symptoms, and a retrieved note. Each cause is marked confirmed or possible, copied from that note. The result includes a short repair suggestion, a ride decision, and the note IDs. When the evidence is insufficient, the agent asks for another photo or a clearer symptom and does not give a repair plan.

- PEAS analysis that defines the performance measure, environment, actuators, and sensors;

- **Performance:** Whether the top cause is in the acceptable set for that case; whether the cited post actually supports that cause; whether dangerous cases are marked do-not-ride; whether the agent stops when it should; whether the gold post appears in the top 3 retrieved results.

- **Environment:** One diagnostic session.
  The input is a photo, year, symptoms, and mileage.
  Forum excerpts are stored locally and do not change during a session.
  Partially observable because the photo does not show internal damage, and the fault is not certain.

- **Sensors:** The user case; the image description; retrieved post IDs and text; the steps already taken

- **Actuators:** describe_image, search_repair_notes, request_more_info, submit_diagnosis. The last two end the session

- The agent's decision boundary, percepts, actions, state, goal or terminal conditions, and interaction with the environment;

- **Decision boundary:** The agent interacts with the environment only through the 4 actuators.
- **Percept:** Current case state plus the latest tool result.

- **Action:** Choose one tool and supply the arguments.
- **State:** stores the bike facts, symptoms, image findings, posts retrieved in this session, candidate causes, safety flags(ride, limited riding, or do not ride), and the number of steps used.

  A session ends in one of three ways: the agent submits a diagnosis with at least one valid citation; if the photo is unusable or the store has no matching fault, it asks for more information; After 6 steps, the session is forced to stop and is counted as a failure.

  A year outside 2013–2018 is outside the scope and should end the session before retrieval. The current program does not check the year.

  The code enforces three rules: a diagnosis cannot be submitted before retrieval, and every cited ID must appear in the posts retrieved in this session, and the submitted status must match that note.

- The selected AI strategy, why it fits the problem, and the principal technical uncertainty that the proof of concept must resolve;

- Strategy: vision model from OpenAI with constrained tools. At each step the model reads the state, chooses one tool, writes the result back into the state, and then decides again. Forum retrieval uses vector similarity. Forum excerpts are embedded ahead of time into a local vector store. The query is the image description, the symptoms, and the bike identity. The search returns the 3 most similar excerpts.

The agent may cite only excerpts the photo and symptoms support, and it must copy each excerpt's status.

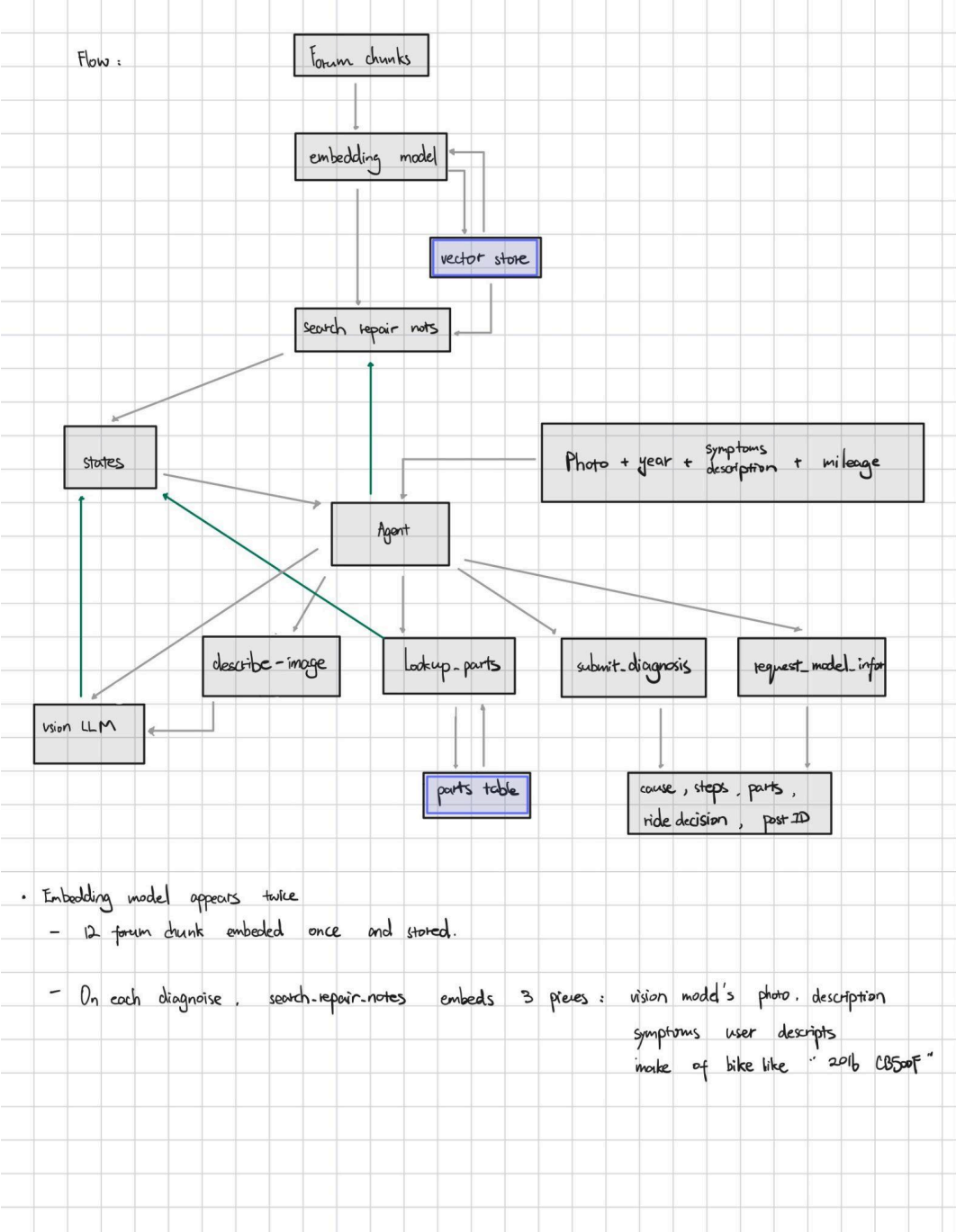
- principal technical uncertainty:

This strategy fits the problem because a fault cannot be settled from one look at a photo. The agent first describes the visible damage, then uses vector search to align the rider’s wording with the wording in the forum. Only after the evidence is sufficient, it submits a plan.

For example, the rider may write “it is wet under the engine,” while the forum post says “coolant at the water-pump weep hole.” Vector retrieval is what connects those two sentences.

The question is: after the vector store returns the top 3 posts, are the agent’s causes actually supported by those posts, or does the agent still assert a fault that the posts do not contain?

- A system diagram showing the major components and information flow; # I delete the lookup parts, not sure going to have it or not. Ignore it on the flow.



- a three-week implementation and evaluation scope, followed by final-week documentation and presentation work;

- Week1: Build the tool loop; select and embed 5 to 8 CB500F forum excerpts; run one real case

- Week2: Add forum excerpts; add safety rules; prepare about 12 labeled cases; write the scoring script; run the same cases once with a single prompt and no tool constraint

- Week3: Run the full evaluation and write the error analysis, report, and presentation slides hopefully

- A repeatable evaluation plan with numerical measures of solution quality, cost, reliability, or another property appropriate to the project;
  - Retrieval hit rate: Share of cases in which the gold post appears in the top 3

  - Cause accuracy: Share of cases in which the top-ranked cause is in the acceptable set

  - Citation precision: Share of citations whose post actually supports the stated cause

  - Safety recall: Share of dangerous cases (brakes, exposed tire cord, fuel leak) marked do-not-ride

  - Abstention accuracy: Share of unusable-photo or out-of-store cases in which the agent correctly stops

  - Invalid-action rate: Share of runs that submit before retrieval, or cite an id that was not returned

  - Cost: Mean tool calls and tokens per case

- Expected limitations, failure conditions, and criteria that would support a final recommendation to `deploy`, `revise`, or `do not use` the system.

The system covers only 2013 - 2018 CB500F faults that are in the store.
A blurred photo does not reveal internal damage.
The output is a pre-ride reference.

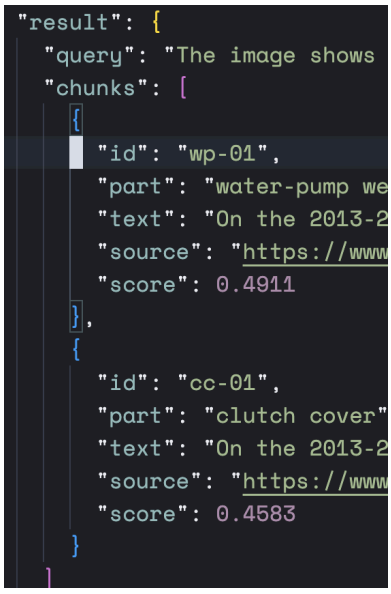
Failures:
The photo is too dark or aimed at the wrong part;
A fault that is absent from the vector store;
A conclusion that the retrieved posts do not support;
A dangerous fault that is cleared to ride.

Criteria:

cause accuracy at least 70 percent, citation precision at least 80 percent, safety recall of 100 percent on the dangerous subset, and zero invalid citations.
Revise: cause accuracy from 50 to 70 percent, or a safety miss that can be fixed by adding posts or tightening the tool constraint.
A dangerous case is still cleared to ride(but not), or citation precision is below 50 percent.

Minimum Proof of Concept:

```
python diagnose.py cases/water-pump.txt
{
  "accepted": true,
  "diagnosis": {
    "causes": [
      {
        "name": "Coolant at the water-pump weep hole",
        "likelihood": "medium",
        "chunk_id": "wp-01"
      },
      {
        "name": "Oil leak from clutch cover gasket",
        "likelihood": "medium",
        "chunk_id": "cc-01"
      }
    ],
    "repair_steps": [
      "Monitor the weep hole for any coolant drips. If dripping occurs, replace the water pump mechanical seal
.",
```



I wrote the 1 chunks.json in the knowledge base just for this minimum proof of concept.

`wp-01` is the water-pump weep hole note.
`cc-01` is the clutch-cover oil-leak note.

Each chunk has an id, a part, a short text, and the URL of the forum post. Nothing in the program downloads or reads those pages. The texts were written to match those posts and stored before the run.

At runtime, each chunk text and the query built from the photo description… will be embedded. The search returns both chunks with scores. The agent may cite only IDs from that result.

In the saved trace, `wp-01` scored 0.4911, `cc-01` scored 0.4583, and the submitted diagnosis cited those IDs. Scores are calculated by cosine similarity, the bigger the number, the closer.

#（That trace matches the two-note store. chunks.json now has 8 notes, so a new run would return the top 3, not those two scores.）